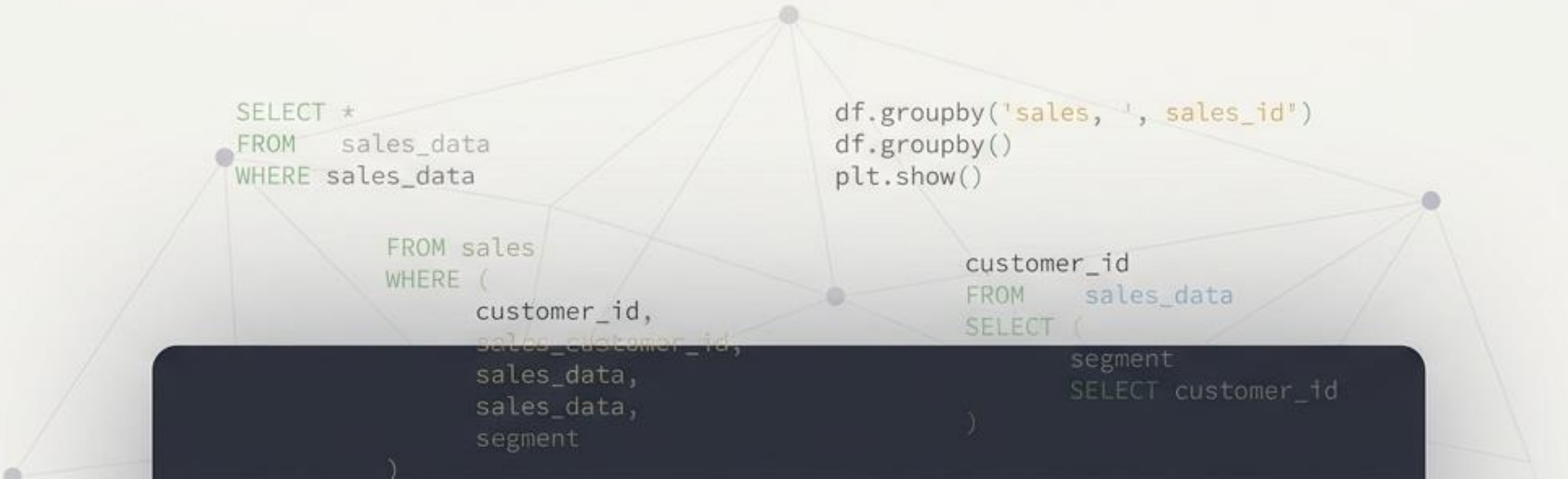


Superstore Analytics Report

An EDA and Customer Segmentation Portfolio using SQL & Python



```
SELECT *  
FROM sales_data  
WHERE sales_data
```

```
FROM sales  
WHERE (  
customer_id,  
sales_customer_id,  
sales_data,  
sales_data,  
segment  
)
```

```
df.groupby('sales', 'sales_id')  
df.groupby()  
plt.show()
```

```
customer_id  
FROM sales_data  
SELECT (  
segment  
SELECT customer_id
```

Environment Setup & Data Ingestion

```
import sql, pyodbc, sqlalchemy
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

%load_ext sql
%sql mssql+pyodbc://               /super_store...

%%sql
SELECT TOP(5) * FROM superstore;
```

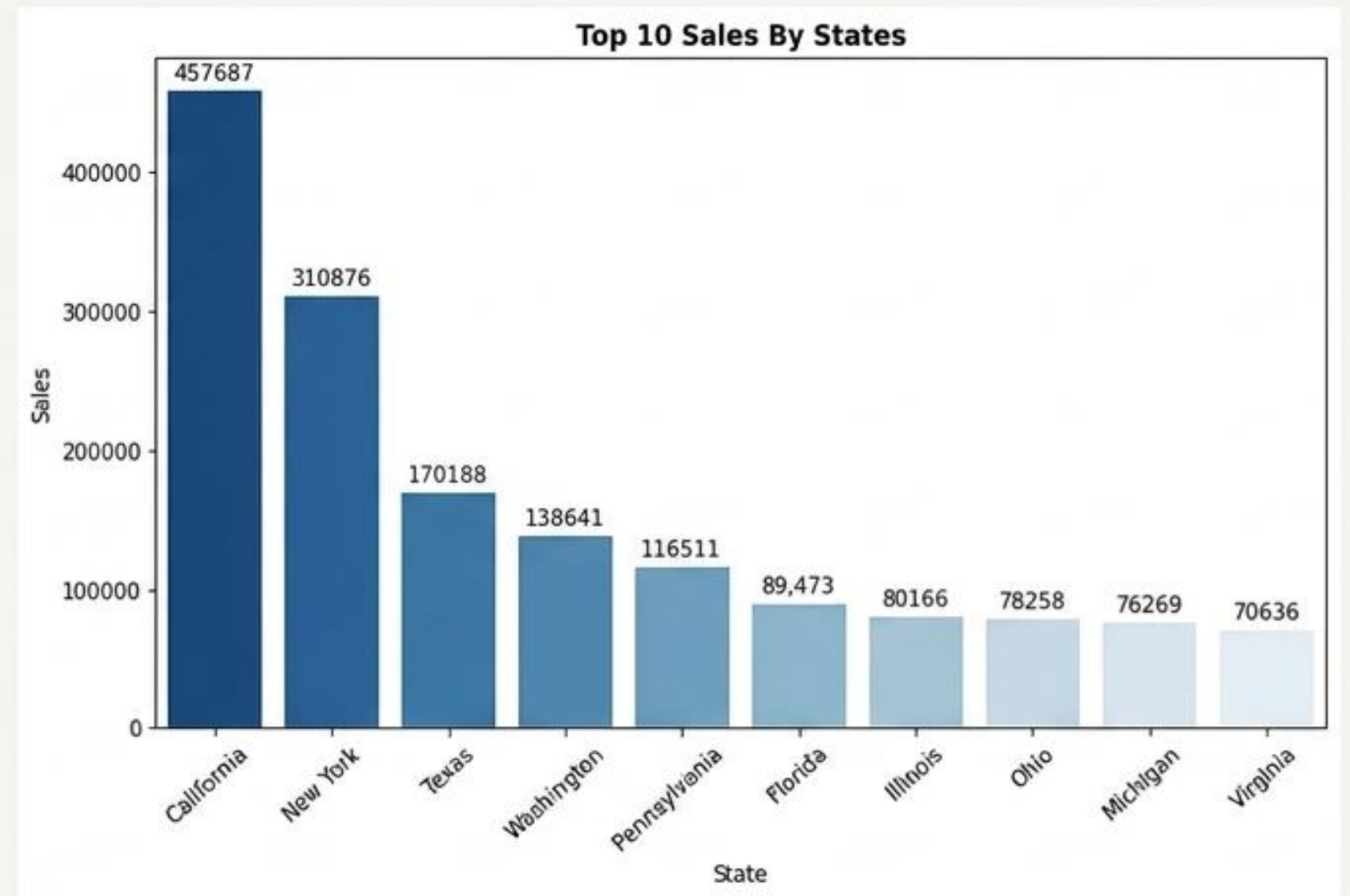
Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	S0-20335	Sean O'Donnell
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	S0-20335	Sean O'Donnell

Geographic EDA: Top 10 Sales by State

```
%%sql state_sales <<
SELECT TOP (10) State, ROUND(SUM(Sales),1) as Sales
FROM superstore
GROUP BY State
ORDER BY Sales DESC;

# sns.barplot execution...
```

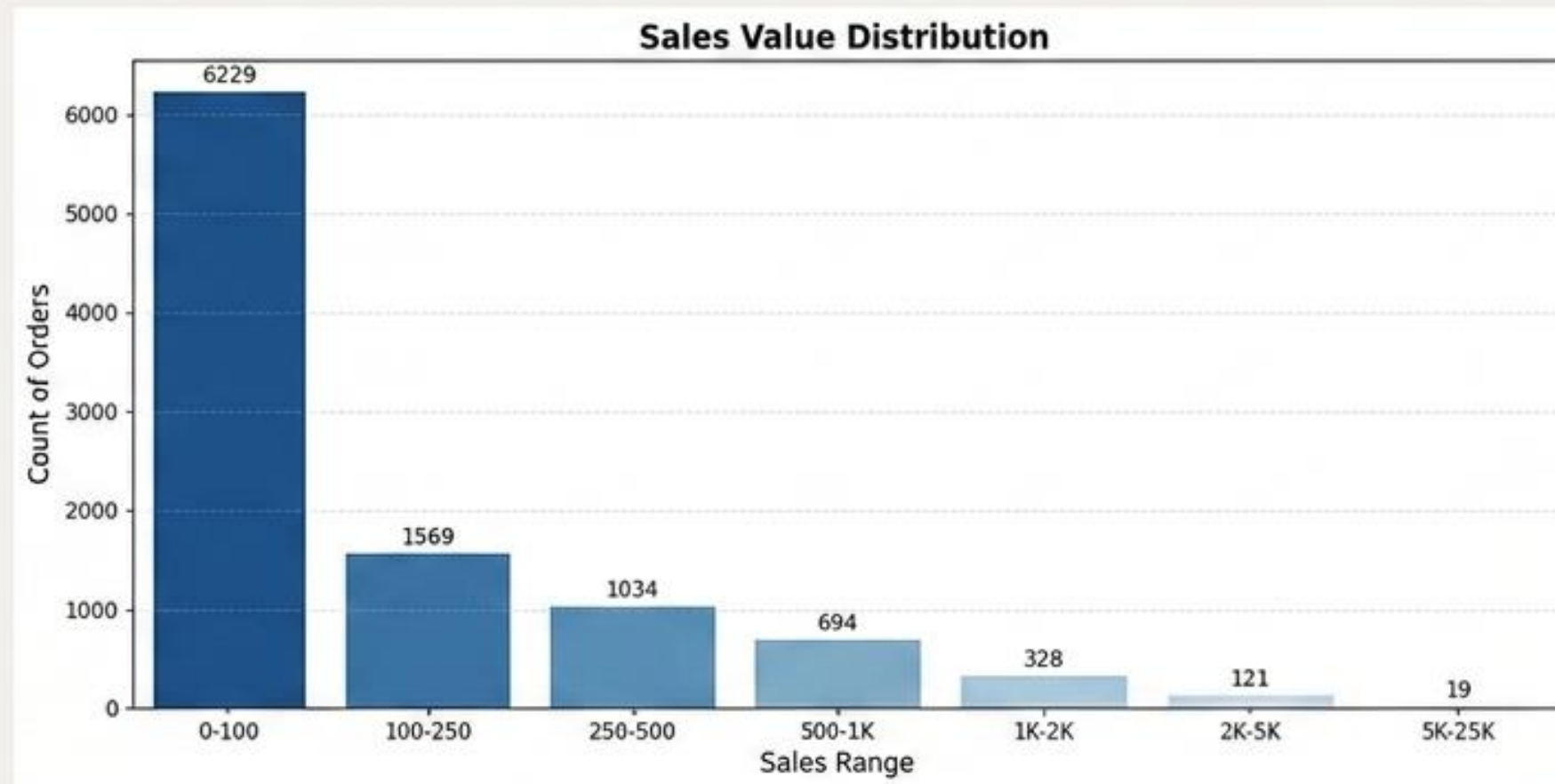
State	Sales
California	457,687.6
New York	310,876.3
Texas	170,188.0
Washington	138,641.3
Pennsylvania	116,511.9
Florida	89,473.7
Illinois	80,166.1
Ohio	78,258.1
Michigan	76,269.6
Virginia	70,636.7



California and New York drive the highest overall sales volume.

Order Value Distribution

```
df_clean = df_sales.DataFrame()
custom_bins = [0, 100, 250, 500, 1000, 2000, 5000, 25000]
bin_labels = ['0-100', '100-250', '250-500', '500-1K', '1K-2K', '2K-5K', '5K-25K']
df_clean['Sales_Group'] = pd.cut(df_clean['Sales'], bins=custom_bins, labels=bin_labels)
# Seaborn barplot execution follows...
```



The vast majority of Superstore orders fall into the \$0–\$100 range, indicating a high-volume, low-AOV (Average Order Value) transactional model.

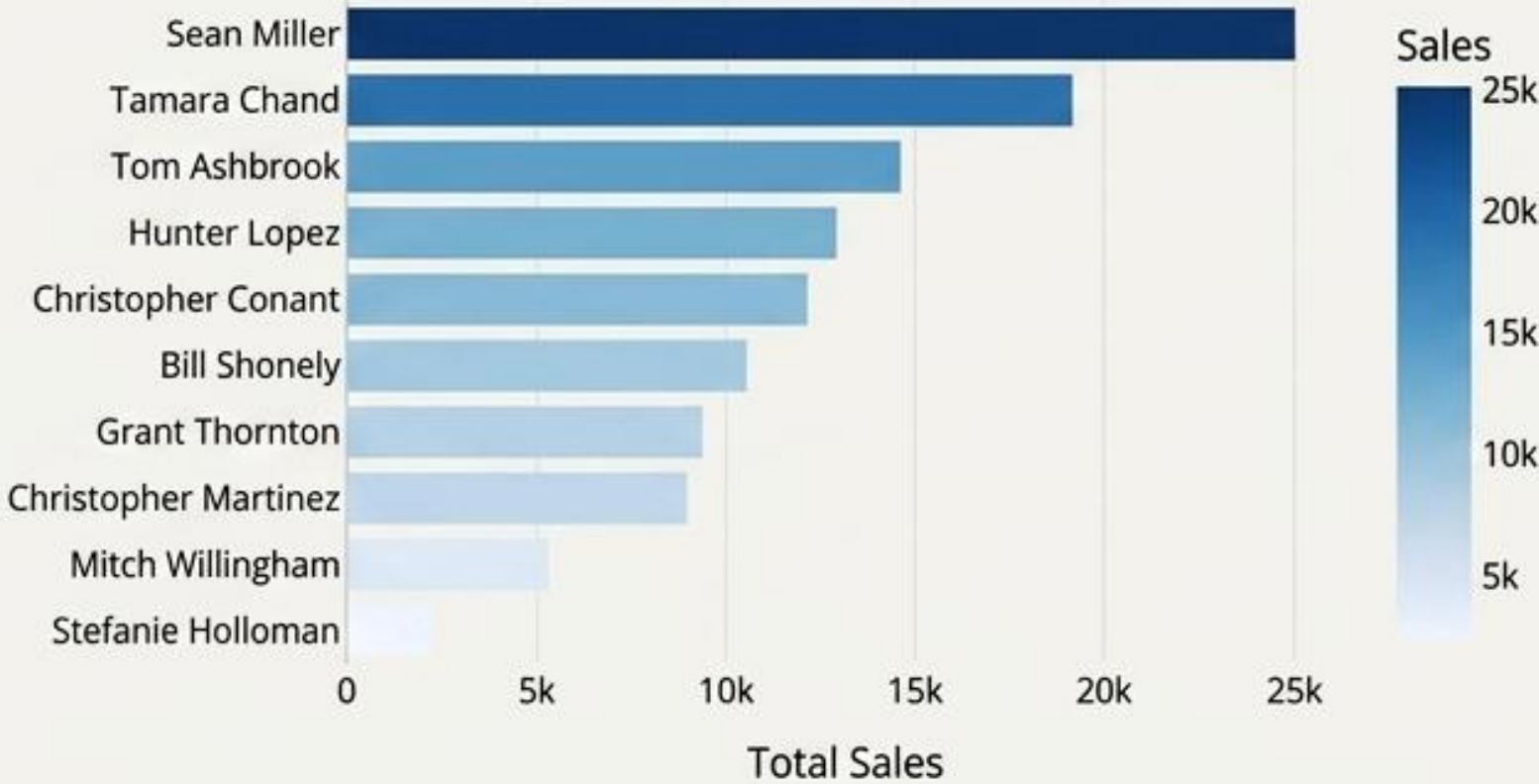
Top Customers Analysis

```
%%sql top_cus <<
SELECT TOP(10) Customer_Name, ROUND(SUM(Sales),1)
as Sales, ROUND(AVG(Sales),1) as Sales_AVG
FROM superstore
GROUP BY Customer_Name
ORDER BY Sales_AVG DESC;

# Plotly horizontal bar chart execution...
```

Customer_Name	Sales	Sales_AVG	Order_Count
Mitch Willingham	5253.9	1751.3	3
Sean Miller	25043.1	1669.5	15
Tamara Chand	19052.2	1587.7	12
Grant Thornton	9351.2	1558.5	6
Tom Ashbrook	14595.6	1459.6	10
Hunter Lopez	12873.3	1170.3	11
Bill Shonely	10501.7	1166.9	9
Christopher Conant	12129.1	1102.6	11
Stefanie Holloman	2096.4	1048.2	2
Christopher Martinez	8954.0	895.4	10

Top Customers Analysis



Sean Miller leads all customers with over \$25K in total sales, sales, despite Mitch Willingham having a slightly higher average order value.

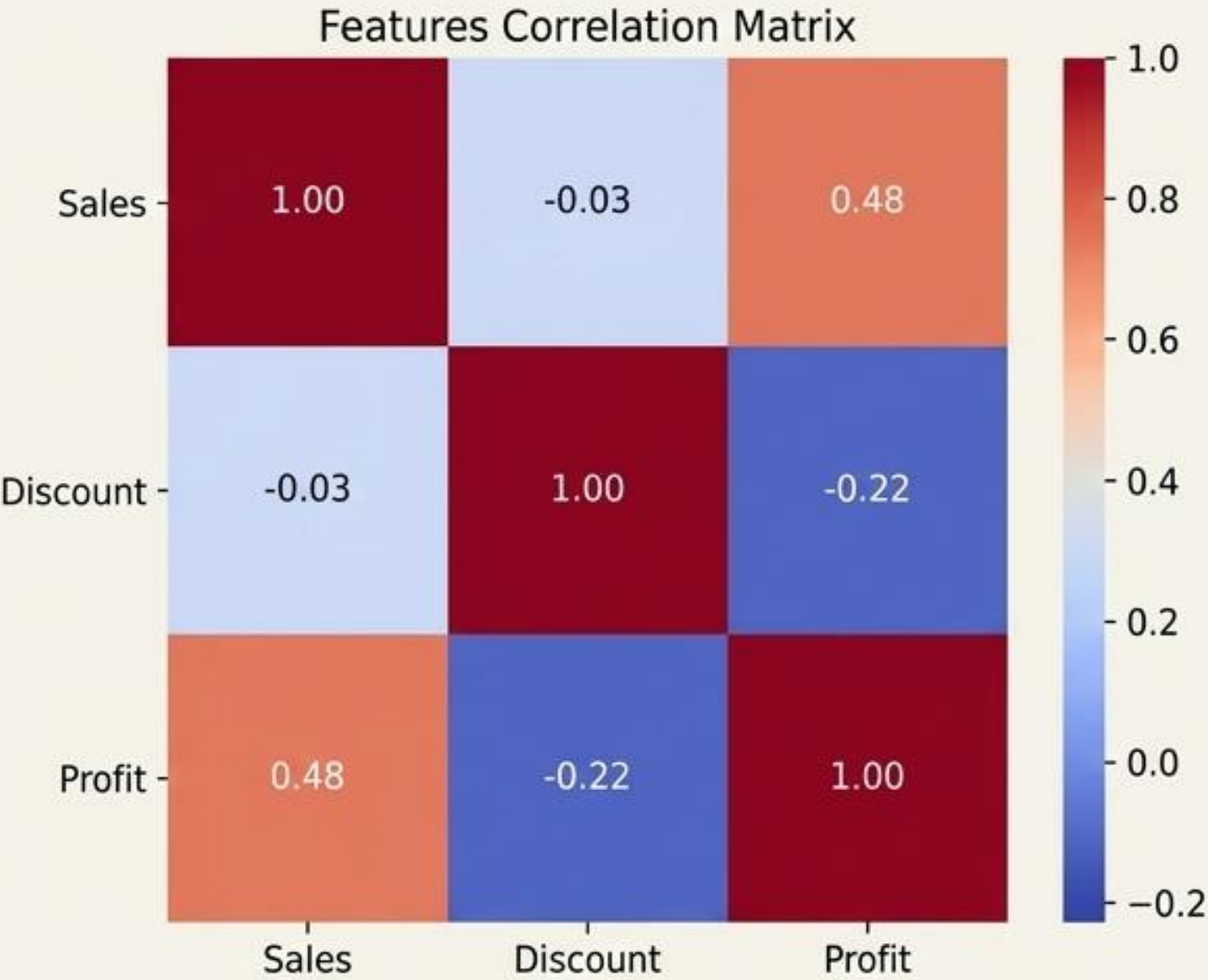
Feature Correlation Matrix

```
%%sql correlation <<
SELECT Sales, Discount, Profit FROM superstore;

corr_matrix = df_clean[['Sales', 'Discount', 'Profit']].corr()
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm")
```

Index	Sales	Discount	Profit
0	261.959991	0.00	41.913601
1	731.940002	0.00	219.582001
2	14.620000	0.00	6.871400
3	957.577515	0.45	-383.031006
4	22.368000	0.20	2.516400
...	...	...	...
9989	25.247999	0.20	4.102800
9990	91.959999	0.00	15.633200
9991	258.575989	0.20	19.393200
9992	29.600000	0.00	13.320000

...remaining rows omitted for readability.

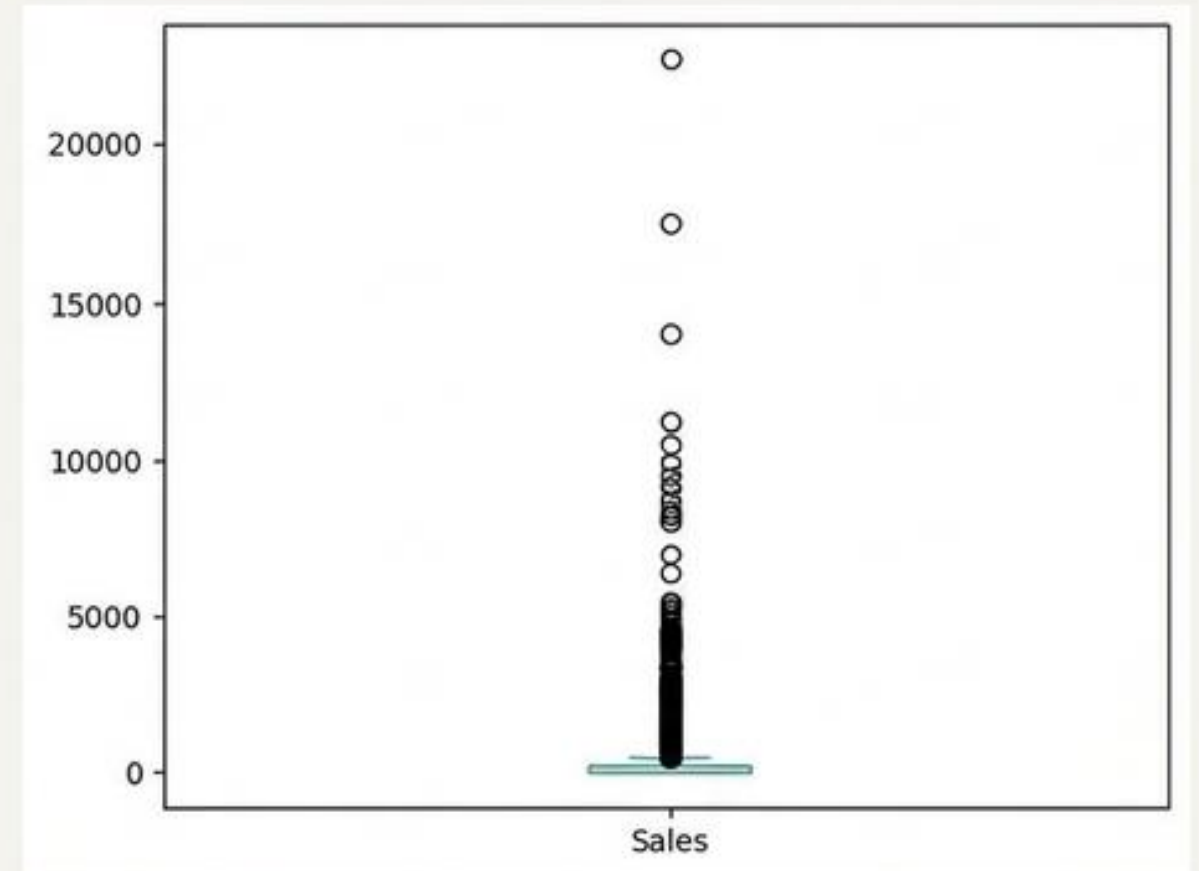


Sales and Profit show a moderate positive correlation (0.48), while Discounts predictably exhibit a negative impact on Profitability (-0.22).

Customer Segmentation Strategy

```
%%sql customer_level <<
SELECT Customer_Name, ROUND(SUM(Sales),0) as Sales,
       ROUND(SUM(Profit),0) as Profit,
       CASE
         WHEN SUM(Sales) > (SELECT ROUND(AVG(Sales),0)
FROM superstore) THEN 'Vip Customer'
         WHEN SUM(Sales) < (SELECT ROUND(AVG(Sales),0)
FROM superstore) THEN 'Normal Customer'
         WHEN SUM(Profit) < 0 THEN
           'Bad Customers'
       END as customer_level
FROM superstore
GROUP BY Customer_Name
ORDER BY Sales DESC;
```

Calculated Median Sales: **\$54.49**



Sales distribution is heavily skewed with extreme outliers. This median and average variance informs our threshold logic for defining VIP segmentation.

Segment Deep-Dive: Identifying "Bad" Customers

```
%%sql
bad_cus <<
SELECT Customer_Name,
       ROUND(SUM(Sales),0) as Sales,
       ROUND(SUM(Profit),0) as Profit
FROM superstore
GROUP BY Customer_Name
HAVING SUM(Profit) < 0
ORDER BY Profit;
```

20.0%

Bad customers make up exactly 20% of the total customer base (155 out of 793 total customers).

	Customer_Name	Sales	Profit
1	Cindy Stewart	5690.0	-6626.0
2	Grant Thornton	9351.0	-4109.0
3	Luke Foster	3931.0	-3584.0
4	Sharelle Roach	3233.0	-3334.0
5	Henry Goldwyn	3248.0	-2798.0
6	Nathan Cano	2219.0	-2205.0
7	Sean Braxton	8058.0	-2083.0
8	Sean Miller	25043.0	-1981.0
9	Christine Phan	5888.0	-1850.0
10	Natalie Fritzler	8323.0	-1696.0

...remaining rows omitted for readability.

Segment Deep-Dive: VIP & Normal Customers

```
%%sql real_segmantation <<
CREATE OR ALTER VIEW r_c_l AS
SELECT Customer_Name, ROUND(SUM(Sales),0) as
      Sales, ROUND(SUM(Profit),0) as Profit,
      CASE ... END as customer_level
FROM superstore
GROUP BY Customer_Name
HAVING SUM(profit) > 0;
```

Final tally yields 616 VIP Customers and 22 Normal Customers (among those with positive profit), serving as the foundational list for targeted marketing workflows.

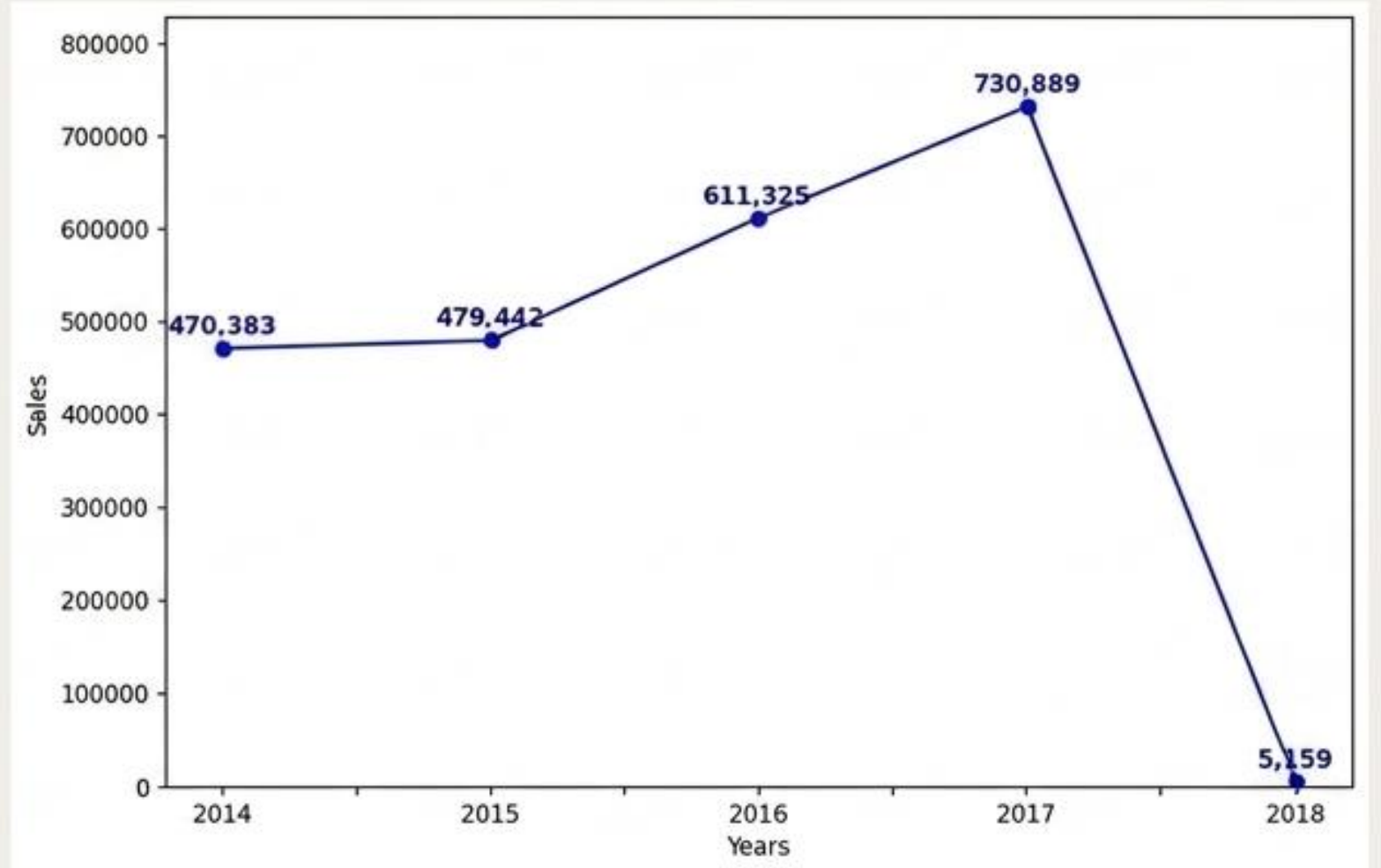
Customer_Name	Sales	Profit	customer_level
Michael Chen	3806.0	698.0	Vip Customer
Brian Moss	7294.0	2199.0	Vip Customer
Tamara Chand	19052.0	8981.0	Vip Customer
Justin MacKendrick	2834.0	754.0	Vip Customer
Doug Bickford	1989.0	439.0	Vip Customer
Jesus Ocampo	1091.0	168.0	Vip Customer
John Huston	529.0	26.0	Vip Customer
Liz Carlisle	2095.0	87.0	Vip Customer
Tracy Collins	743.0	218.0	Vip Customer
Eugene Moren	4588.0	1319.0	Vip Customer

...remaining rows omitted for readability.

Temporal Trends: Sales Over Time

```
%sql sales_year <<
SELECT YEAR(Ship_Date) as Years, SUM(Sales) as Sales
FROM superstore
GROUP BY YEAR(Ship_Date)
ORDER BY YEAR(Ship_Date);
```

Years	Sales
2014	470,383
2015	479,442
2016	611,325
2017	730,889
2018	5,159



Steady YoY growth peaked at \$730,889 in 2017 before a sharp, anomalous decline to \$5,159 in 2018. This likely indicates an incomplete dataset for 2018, requiring further validation before finalizing annual reports.